

The things only you can do

Everything here needs an account in your name, a decision that is yours, or a human holding a phone. Nothing on this list can be done for you.

BEFORE ANYTHING ELSE — YOU HAVE TWO SUPABASE PROJECTS

The project in your screenshot is **hedaklvumeihfsgokdsi** — brand new, empty, no migrations, no functions. Everything built so far points at **jnxvwdcvnwigowafdxvl** — which is still running, and still holding whatever enquiries have come in.

Two live projects is how data gets lost. Pick one before deploying anything, or you will end up with customers split across both and no single place to look.

Task 0 — decide which project

OPTION	WHAT IT MEANS	MY VIEW
Keep the OLD one jnxvwdcv...	Nothing to change. Already has your existing enquiries and eight working functions.	Recommended. Least risk, and nothing is lost.
Move to the NEW one hedaklvu...	Cleaner name, tied to contact@alsaitigrowth.com. But existing enquiries stay behind unless they are exported and re-imported.	Only if the old project is going away for a billing or ownership reason.

If you choose the new one, the switch is one command — the reference used to be hardcoded in twelve places, and I have centralised it:

```
node tools/set-project.js hedaklvumeihfsgokdsi sb_publishable_YOURNEWKEY
```

The new publishable key is in **Project Settings** → **API**. It is safe in a browser and safe in git — the tool refuses anything else.

Do these first — free, about an hour

Together these finish the lead pipeline. After them, no enquiry can be lost.

TASK 1 5 minutes

Connect the command line

This is what lets the deployment happen at all. It opens your browser; you approve; the access token is stored on the computer, not shared with anyone.

```
npm install -g supabase
supabase login
supabase link --project-ref YOUR_CHOSEN_REF
```

THIS IS ALSO WHAT WOULD LET ME HELP

Once you have run `supabase login` on that machine, I can run the deploy and secret commands through your authenticated session — without ever seeing the token. That covers roughly eight of the thirteen finish-line items. What I will never do is take a key, password or token through a chat message.

TASK 2 10 minutes

Deploy the database and the functions

Database first. The new contact form writes columns the migrations create; deploy the functions first and every submission fails.

```
supabase db push
supabase functions deploy contact-submit lead-notify health events-consume
node tests/verify-deploy.js
```

The last command needs no password and checks everything from outside. It should end with `passed: 25` and no failures.

TASK 3 2 minutes

Set three values

You have added the Resend key already. On its own it sends nothing.

```
supabase secrets set \
  ALERT_FROM_EMAIL="Alsaiti Growth " \
  LEAD_NOTIFICATION_TO="contact@alsaitigrowth.com" \
  EVENTS_CONSUMER_SECRET="$(openssl rand -hex 24)"
```

Write that consumer secret down — task 5 needs it.

TASK 4 15 min + DNS wait

Verify your sending domain

At resend.com/domains, add **alsaitigrowth.com** and pick the **eu-west-1** region. It generates four records to add in Cloudflare. Until this shows **Verified**, Resend delivers only to your own address — alerts to anyone else fail silently.

TYPE	NAME	VALUE
MX	send	feedback-smtp.eu-west-1.amazonses.com (priority 10)
TXT	send	v=spf1 include:amazonses.com ~all
TXT	resend._domainkey	copy from your dashboard – about 400 characters, must not wrap
TXT	_dmarc	already there – leave it

TASK 5 10 minutes

Switch on the scheduler

Supabase → Database → Extensions → enable **pg_cron** and **pg_net**. Then re-run `supabase db push`, and in the SQL editor schedule the job that actually delivers your lead alerts:

```
select cron.schedule('alsaiti_events','* * * * *', $$
select net.http_post(
url := 'https://YOUR_REF.supabase.co/functions/v1/events-consume',
headers := jsonb_build_object('Content-Type', 'application/json',
'x-consumer-secret', 'YOUR_SECRET_FROM_TASK_3'),
body := '{}':jsonb)
$$);

select jobname, schedule from cron.job; -- expect two rows
```

No rows means nothing delivers your alerts and nothing enforces the deletion your privacy policy promises.

TASK 6 5 minutes

Point a monitor at the health check

Better Stack, Cronitor or UptimeRobot — any of them, free tier. Five-minute interval, alert on any non-2xx. The endpoint returns 503 when degraded, so nothing else needs configuring.

```
https://YOUR_REF.supabase.co/functions/v1/health
```

Without this, a stalled queue is invisible: leads keep saving, the dashboard keeps working, and nobody is told about any of them.

TASK 7 5 minutes

Create a second test account

Supabase → Authentication → Users → Add user. Any email will do. This exists so isolation between customers can be *proven* rather than assumed — send me the two logins and I will run the test and send you the result.

TASK 8 5 minutes

Two small clean-ups

In the SQL editor, remove the junk row from when I was testing whether the endpoint validated email addresses:

```
delete from contact_submissions where email = 'notanemail';
```

And change the demo password. It sits in a public file so changing it does not hide it — what matters is that the demo account cannot reach real customer data.

Then these — each needs an account in your name

None of these can start until the account exists. That is the schedule, not the engineering.

#	TASK	WHAT IT UNLOCKS	COST
9	HubSpot developer app <code>developers.hubspot.com</code>	The Integrations screen stops saying Needs setup. The connector is written and waiting.	Free
10	Telnyx account, verified and funded	A real phone number you can connect. All four functions are written and deployed.	Number rental + call charges
11	LiveKit account + somewhere always-on	The AI answering real calls. The worker is built and its logic tested.	Hosting + per-minute AI
12	A phone number a human answers	Where urgent calls transfer to when the AI escalates.	Free

FOR TASK 9 — REGISTER THIS EXACTLY

`https://YOUR_REF.supabase.co/functions/v1/crm-callback`

Character for character. HubSpot rejects the entire installation if the redirect URL differs by so much as a trailing slash.

FOR TASK 10 — THE SECOND KEY MATTERS

You need **both** `TELNYX_API_KEY` and `TELNYX_PUBLIC_KEY`. Without the second, webhook signatures cannot be verified — and unverified webhooks are refused, so the number simply never answers and nothing obvious explains why.

And these need a person, not a system

#	TASK	WHY IT MATTERS
13	A solicitor reads the privacy policy and terms	They were written to match what the system actually does. That is accuracy, not legal advice, and they now carry retention commitments and a liability cap.
14	A native speaker checks the Spanish and Arabic	Those languages now contain legal text. A machine-translated liability clause is a genuine risk.
15	Open the site on a real iPhone	Safari's audio rules are the one thing no browser test or emulator reproduces, and the voice feature depends on them entirely.
16	Decide: call recording on or off, and for how long	This changes what must be stored and what callers must be told. Answer it before the phone work starts, not during.

IF YOU ONLY DO ONE THING

Tasks 0 to 2. Decide the project, connect the command line, deploy. That is about twenty minutes and it is the difference between enquiries landing somewhere nobody looks and enquiries reaching you.

Everything else in this document depends on it, and none of the work already built protects a single customer until it runs.

Never send an API key, password, access token or recovery code through chat, email or a screenshot — including to a developer. Enter them directly into the provider dashboard, or with `supabase secrets set` on your own machine.